Data Types & Variables in C

Data Types

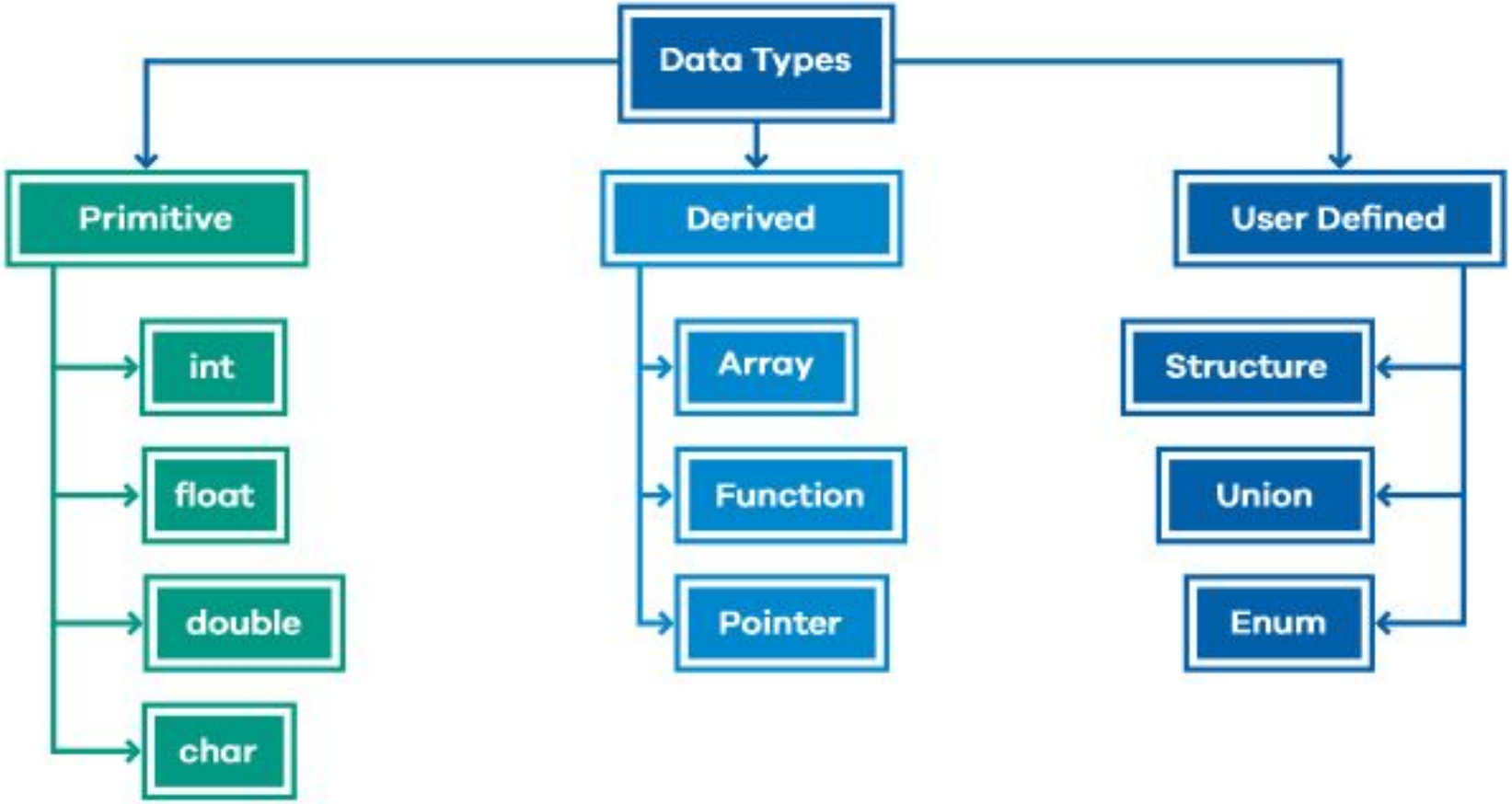
- Computer processes certain kinds of ***data*** consisting of **numbers**, **characters** and **strings** and provides useful **output** called ***information***.
- The task of processing of data is accomplished by executing a sequence of precise instructions called ***program***.
- The instructions constituting the program represent some kind of **operations**, which the computer can perform.
- To specify an operation we need ***operators*** and ***operands***.
- In data processing, **data** plays the role of **operands** and it may be numeric and/or non-numeric.
- C classifies these data into certain categories, called ***data types*** so that the operations performed on them are specific.

Contd…

- C provides a predefined set of data types for handling the data it uses.
- When a program is written, there may be instructions to input data, which needs prior memory allocation.
- Hence the program needs a means to identify the data so that it can tell the compiler regarding the requirement of memory space and the operations that can be performed on these.
- C **data types** are classified into various categories.

1. **Primitive Data Types**
2. **Derived Types**
3. **User Defined Data Types**

Data Types in C



Primitive Data Types

- **Primitive data types**, also known as **fundamental** or **basic** or **built-in** data types**.**
- There are 5 primitive data types:

  int **char** **float** **double** **void**

- The **int** data type represents the **integers**.
- **char** data type represents all **character constants**.
- The data belonging to **char** type are **internally stored** as **integers** as the **letters**, **symbols** etc are represented in memory by associated number codes (called **ASCII codes**, which are numbers ranging from **0 to 255**)
- The **float** data type represents the **real (floating-point) numbers**.
- The **double** data type is also used for handling **floating point numbers**. But it differs from float type in the sense that the data represented by **double** takes twice as **much memory** as type **float.**

Contd…

- The **double** stores floating point numbers with much larger range and precision (significant number of digits after the decimal point)
- it stands for **double precision floating point**. Through the double is larger, it is slower in the case of accessing the data
- The last primitive type named **void** is special in the sense that it represents a **null set of data** or an **empty set of data**.
- It is the type name of the data that does not exist (or cannot express)

Uses of Data Types

- When program is written in C, we will make use of **identifiers** to **specify** or refer **data**.
- Being a language C has certain **rules** or **conventions** that are to be **strictly followed** while writing programs.
- This gives the **importance** to **data types**.
- Thus the major uses of data types can be listed as follows:
  - As the **type specifier** for the **data** and **functions** in the program.
  - To **declare variables** and **functions**.
  - For the **explicit type conversion** of data.

Data Type Modifiers

- **Type modifiers** are the keywords used along with the basic data types to extend the range of data handled by the basic types.
- This is achieved by excluding the negative numbers in the range and/or by allocating more number of bytes for the data.
- The keyword **signed, unsigned, long** and **short** are the **type modifiers** of C.
- The basic data types have many combinations with these modifiers preceding them.
- But the data types **void** and **float** have **no modifiers**.
- Also note that **double** is not a modifier of **float**.

Table 9. 5 Memory allocation for Fundamental data types

Data type	Space in memory		Range of value
	in terms of bytes	in terms of bits	
char	1 byte	8 bits	-127 to 128
int	2 bytes	16 bits	-32,768 to 32,767
float	4 bytes	32 bits	3.4×10^{-38} to 3.4×10^{38} -1
double	8 bytes	64 bits	1.7×10^{-308} to 1.7×10^{308}-1

Table 9.8 Memory allocation for floating point Data types

Data type		Space in memory		Range of value
		in terms of bytes	in terms of bits	
float	signed fractional value	4 bytes	32 bits	3.4×10^{-38} to 3.4×10^{38}-1
double	signed more precision fractional value	8 bytes	64 bits	1.7×10^{-308} to 1.7×10^{308} -1
long double	signed more precision fractional value	10 bytes	80 bits	3.4×10^{-4932} to 1.1×10^{4932} -1

Table 9.6 Memory allocation for Data types

Data type		Space in memory		Range of value
		in terms of bytes	in terms of bits	
short	short is a short name for **short int**	2 bytes	16 bits	-32,768 to 32,767
unsigned short	an integer number without minus sign.	2 bytes	16 bits	0 to 65535
signed short	An integer number with minus sign	4 bytes	32 bits	-32,768 to 32,767
Both short and signed short are similar				
int	An integer may or may not be signed	2 bytes	16 bits	-32,768 to 32,767
unsigned int	An integer without any sign (minus symbol)	2 bytes	16 bits	0 to 65535
signed int	An integer with sign	2 bytes	16 bits	-32,768 to 32,767
Both short and int are similar				
long	long is short name for long int	4 bytes	32 bits	-2147483648 to 2147483647
unsigned long	A double spaced integer without any sign	4 bytes	32 bits	0 to 4,294,967,295
signed long	A double spaced integer with sign	4 bytes	32 bits	-2147483648 to 2147483647

Derived Data Types

- **Data types** derived from the basic data types are called derived data types.
- The following are the derived data types:

1. **Arrays**: They refer to a named set of finite number of same type of data, each of which can be referenced by mentioning its position (called subscript) in the list. There are **one-dimensional** and **multi-dimensional arrays**.

2. **Function**: It is the named unit of a set of instructions to perform a particular task. A function is invoked from other parts of the program (say some other function). A function should be declared before it is defined if the function calling statement appears before its definition.

3. **Pointer:** A variable that can hold the address of a memory location. The declaration of the pointer consists of a data type, an * (asterisk) and the variable name. The data type of the pointer depends upon the type of the data that is pointed to by it.

Identifiers

- Identifiers are the words that identify different program elements such as memory locations (**variables**), **functions** (function names), **statements** (label) etc.
- Each language has its own rules for naming these identifiers.
- The following rules are common:
  - **Identifiers is an arbitrary long sequence of letters and digits.**
  - **The first character must be a letter.**
  - **The underscore (_) counts as a letter.**
  - **White spaces are not allowed.**
  - **Uppercase and lowercase letters are different.**
  - **Keywords cannot be used as identifiers.**

Contd…

- The following are some of **valid identifiers:**

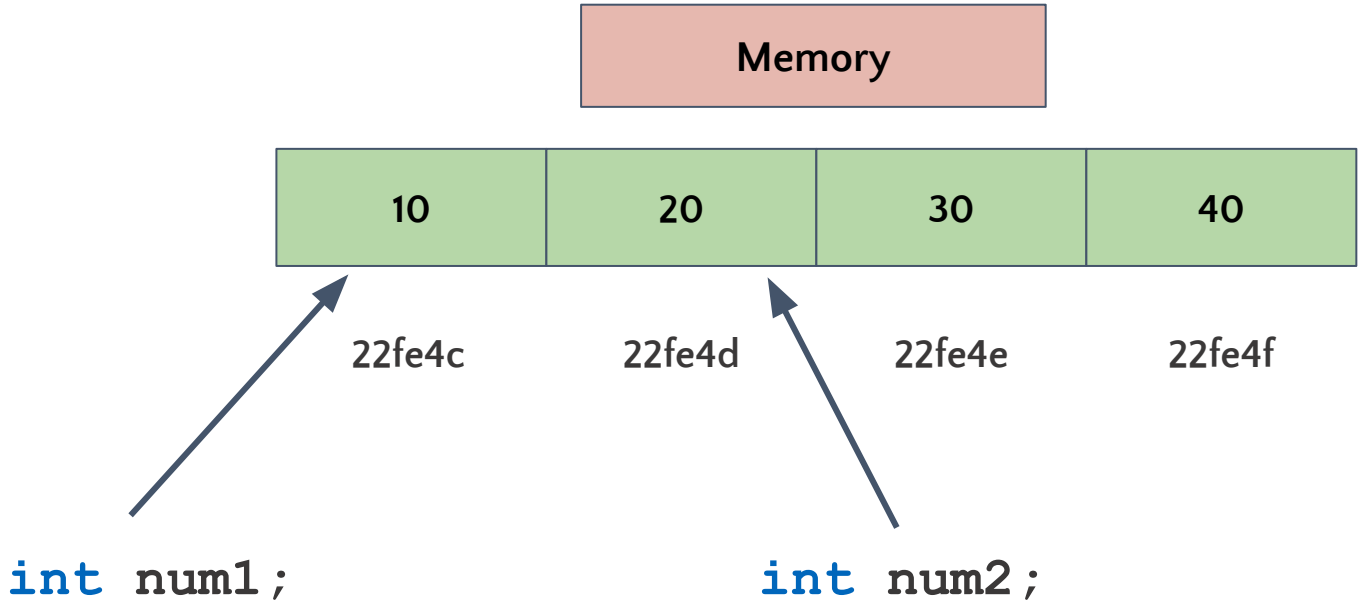
Num	**_avg_hgt**	**Program**
var	**addr2**	**Int**
realnum	**FILE**	

- The following are **invalid identifiers:**

Stud-rec	The hyphen character **-** is used.
2number	Digit is used as first character.
int	Keyword (already reserved by the language)

Contd...

- While programming, we very often need to access the memory to read and write data.
- The memory addresses are hexadecimal values.
- So, it is very hard to keep track of hexadecimal values while programming.
- To manipulate memory address easily, attach a variable name to the memory address.
- So, it will be very easier to manipulate the memory address using a variable name rather than hexadecimal values.



Contd...

- In the previous diagram, **memory address 22fe4c** has **mapped** with the **variable name num1.**
- Using **variable num1**, we can easily manipulate the **memory address 22fe4c**.
- Similarly, using **variable num2**, we can easily manipulate the **memory address 22fe4d**.

Valid Variable names in C

1. Variable name should start with **letter (a-z,A-Z)** or **underscore (_).**

Valid	Invalid
age	1age
_age	
Age	

2. In variable name, **no special characters allowed** other than underscore (_).

Valid	Invalid
_age	age_*
age_	+age

Contd…

3. Variables are **case sensitive.**

age and **Age** are different, since variable names are case sensitive.

4. Variable name can be **constructed with digits and letters**.

Example

Age1

Age2

5. Variable name should not be a **C keyword**. **Keywords** are also called as **reserved words**.

Example

int, **main** are reserved for special meaning. So, we should not declare **keyword** as a **variable name.**

Data Types in C

- Assume that we are going to use our **age** in the program.
- To store and access age in the program, it's mandatory to define the data type of age first.
- Let's choose the **data type** for age.

How to select the data type for age?

- We always represent our **age** using a whole number or an integer. Say **20, 25. Floating number** is not used to represent our **age**. Like 20.3 years' old. So, **integer data type** should be used for age.
- similarly to represent weight, we always represent our **weight** using **floating number**.

  Example **52.3 kg, 70.5 kg** etc.

- Use the float data type for weight.
- Define the data type of a variable (age, weight) before assigning a value to it.

Contd…

Major data types in C

Data Type	Description	Example	Size
int	To store an integer value	-5, -1, 0, 4	4 bytes
float	To store a floating point number or number with fractional parts	10.3, 13.56, -2.45, 1.414	4 bytes
double	Same as float but bigger space to store number	123456.34534, -234345345.244466 etc	8 bytes
char	To store a single character	'a', 'b', 'c', '*', '#'	1 byte
void	It is is used to specify that no value is present.	void fun() (i.e) fun() will not return anything int main(void) (i.e) main will not take argument	1 byte

Keywords in C

- Keywords are the words (tokens) that convey specific meaning to the compiler.
- aka **reserved words** because they are reserved by the language for special purposes and cannot be redefined for any other purpose.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	Volatile
const	float	short	Unsigned

Printing Variables

- Using printf function, we can print the value of a variable.
- In order to print the variable value, we must instruct the printf function the following details,
    1. specify the **format of variable** 2. **variable name** which we want to print.

Example:

```
#include<stdio.h>
int main()
{
    int  i = 10;

    printf("Value of i = %d\n",i);



    return 0;
}
```

Basic Format Specifiers in C

Format Specifier	Data Type	description	Syntax
%d	int	To print the integer value	`printf("%d",<int_variable>);`
%f	float	To print the floating number	`printf("%f",<float_variable>);`
%lf	double	To print the double precision floating number or long float	`printf("%lf",<double_variable>);`
%c	char	To print the character value	`printf("%c",<char_variable>);`

Tasks

1. Write a program in C to enter 2 numbers and find its sum. (two_sum.c)
2. Write a program in C to enter 3 numbers and find its sum. (three_sum.c)
3. Write a program in C to enter 2 numbers and find its sum and average. (sum_average.c)
4. Write a program in C to enter a number and find its square and cube. (square_cube.c)
5. Write a program in C to enter the length and breadth of a rectangle and calculate its perimeter and area.(peri_area.c)